Ejercicio U03.E05: “Motor recomendador reproducible”.

«Recomendar no es predecir estrellas: del acierto medido a la decisión que no encierra al usuario en lo más popular»

Por:
Melvin H. Rivas Genao
Matrícula 100279298

Ejercicio Presentado a la
Escuela de Informática, Facultad de Ciencias, División de Postgrado y Educación Permanente, Como Cumplimiento del Curso “Ciencia de Datos II” (INF-8239),
Unidad 03: Algoritmos y Características Generativas,
Maestría en Ciencia de Datos e Inteligencia Artificial,
Edwin Ramón José Nolasco.

Universidad Autónoma de Santo Domingo (UASD)
Santo Domingo de Guzmán, D.N.
Noviembre de 2026

Contenido

Ejercicio U03.E05: Motor recomendador reproducible. Consolidación de LAB08 y LAB09 sobre MovieLens....1
Objetivo....1
Alcance del trabajo....1
Introducción....2
Datos, licencia y auditoría....2
Adquisición reproducible y términos de uso....2
Auditoría del conjunto de datos....3
Particiones y corte temporal....4
Leave-last-out temporal por usuario....4
Simulación de usuarios fríos....4
Modelos....5
Líneas base....5
Recomendador por contenido....5
Factorización matricial....6
Híbrido y política de fallback....6
Evaluación y decisión....6
Métricas....6
Selección en validación y frontera de Pareto....7
Resultados en el conjunto de prueba....9
Comparación pareada....10
Ratings frente a ranking....10
Usuarios con historial pequeño y amplio....11
Usuarios fríos....11
Tres perfiles....12
Costo computacional (Green AI)....12
Errores corregidos y límites....13
Errores detectados durante el desarrollo....13
Límites del estudio....13
Riesgos y mitigación....14
Pruebas, reproducibilidad y repositorio....14
Declaración de uso de inteligencia artificial....15
Conclusiones....16
Referencias....17
Apéndice. Repositorio del proyecto....19

Ejercicio U03.E05: Motor recomendador reproducible. Consolidación de LAB08 y LAB09 sobre MovieLens

Objetivo

Consolidar de forma individual las evidencias de los laboratorios LAB08 y LAB09 en un producto ejecutable y defendible: un motor que recomiende películas a partir de valoraciones previas, que pueda reproducirse desde cero y cuyas decisiones queden justificadas con evidencia.

El ejercicio busca demostrar que se puede recorrer el ciclo completo de un sistema de recomendación: obtener y auditar los datos respetando su licencia, comparar enfoques de distinta naturaleza bajo un mismo protocolo, medir no solo el acierto sino también la cobertura y el costo, y documentar con honestidad los errores y los límites.

Alcance del trabajo

El mandato del ejercicio exige los siguientes entregables, todos incluidos en el repositorio del proyecto:

- Un documento breve con enlace al repositorio y la tabla principal de resultados (este informe).
- Notebook ejecutado, código fuente (*src*), pruebas automáticas (*tests*), reportes (*reports*), *requirements.txt* y README.
- Datos reproducibles con respeto de los términos de uso, sin redistribuir MovieLens.
- Explicación de las líneas base, las particiones, los errores, los límites y la decisión final.
- Declaración de las herramientas de inteligencia artificial utilizadas, los prompts relevantes, las verificaciones y las correcciones.

Introducción

Un sistema de recomendación ordena un catálogo para cada persona. El problema parece sencillo, pero tiene dos trampas que condicionan todo el trabajo. La primera es que los datos son muy dispersos: cada persona valora una fracción mínima del catálogo y la ausencia de una valoración no significa rechazo, solo que esa combinación no se observó. La segunda es que medir bien un recomendador no es lo mismo que predecir bien una calificación: un modelo puede estimar con precisión cuántas estrellas daría alguien a una película y, aun así, ordenar mal lo que esa persona verá después (Cremonesi et al., 2010).

Por esa razón el trabajo no se limita a buscar el modelo con mejor métrica. Compara cuatro familias de enfoques, popularidad, contenido, factorización matricial y un híbrido, bajo el mismo corte temporal; mide el acierto del Top-10 junto con la cobertura del catálogo y la concentración en los títulos más populares; y elige la configuración final con una regla de Pareto que también considera el costo computacional, en la línea de la propuesta de Green AI (Schwartz et al., 2020).

El documento comienza con los datos, su licencia y la auditoría. Luego describe las particiones y los modelos, presenta la selección en validación y los resultados en prueba, incluyendo usuarios con historial pequeño, usuarios nuevos y tres perfiles concretos. Finalmente expone los errores corregidos durante el desarrollo, los límites del estudio, la reproducibilidad, la declaración de uso de inteligencia artificial y la decisión final.

Datos, licencia y auditoría

Adquisición reproducible y términos de uso

Se utilizó **MovieLens Latest Small**, publicado por GroupLens Research de la Universidad de Minnesota (Harper y Konstan, 2015). Contiene 100 836 valoraciones de 610 usuarios sobre 9 742 películas, en una escala de 0.5 a 5 estrellas, registradas entre 1996 y 2018. Su README oficial permite el uso con fines de investigación no comercial, exige citar a Harper y Konstan (2015) y prohíbe redistribuir los datos sin permiso.

En consecuencia, el repositorio **no contiene MovieLens**: la carpeta *data/raw/* está excluida en *.gitignore*. El script *download_data.py* descarga el archivo oficial o instala un zip descargado

manualmente, y registra en un manifiesto la URL, la fecha y la huella SHA-256 del archivo y de cada CSV. La huella del zip utilizado es *696d65a3…c9e436*; cualquier persona puede comprobar que trabaja con los mismos bytes. GroupLens advierte que esta versión es de desarrollo y puede cambiar, por lo que los resultados valen para esa huella y no deben compararse sin más con artículos publicados.

Auditoría del conjunto de datos

La función *audit()* verifica un contrato antes de usar los datos: columnas esperadas, ausencia de nulos, identificador de película único, ausencia de pares usuario–película duplicados, claves foráneas entre valoraciones y películas, rango y paso de la calificación y marcas de tiempo positivas. Todas las comprobaciones pasaron. La Tabla 1 resume los descriptivos más relevantes.

Tabla 1

Descriptivos de MovieLens Latest Small

Medida	Valor
Usuarios / películas en catálogo / películas valoradas	610 / 9 742 / 9 724
Valoraciones (densidad de la matriz)	100 836 (1.70 %)
Rating medio	3.50
Valoraciones por usuario: mínimo / mediana	20 / 70.5
Películas con una sola valoración	3 446
Cuota de valoraciones del 10 % de películas más valoradas	60.0 %
Periodo	1996-03-29 a 2018-09-24
Películas sin género	34

Nota. Salida de *scripts/audit_data.py*.

Dos hallazgos marcan el resto del trabajo. El primero es la **cola larga**: el 10 % de las películas concentra el 60 % de las valoraciones y más de un tercio del catálogo valorado tiene un solo voto. El segundo es que todas las personas tienen al menos 20 valoraciones por diseño del dataset; no existen usuarios realmente nuevos, por lo que el arranque en frío tuvo que simularse. Tampoco hay atributos demográficos, de modo que no es posible evaluar equidad por grupos de población.

Particiones y corte temporal

Leave-last-out temporal por usuario

Las valoraciones de cada persona se ordenaron por marca de tiempo. La **última** se reservó para prueba, la **penúltima** para validación y el resto para entrenamiento (Tabla 2). Así el modelo nunca aprende del futuro de una persona para explicar su pasado; una prueba automática verifica que ninguna valoración de entrenamiento sea posterior a las de validación y prueba del mismo usuario.

Tabla 2

Tamaño de las particiones

Partición	Valoraciones	Usuarios
entrenamiento	99 616	610
validación (penúltima)	610	610
prueba (última)	610	610

Toda decisión de modelado, factores, épocas y pesos del híbrido, se tomó en validación con tres semillas (42, 7 y 2025). Después se reentrenó con entrenamiento y validación juntos y se evaluó **una sola vez** en prueba. El ranking se calcula sobre todo el catálogo no visto por cada persona, sin muestrear negativos.

El corte tiene un límite que conviene declarar: es temporal por usuario y no global, de modo que el entrenamiento puede contener valoraciones de otras personas posteriores al momento de prueba de alguien. Además, el 3.8 % de los ítems de prueba no tiene ninguna valoración de entrenamiento, por lo que ningún modelo colaborativo puede acertarlos.

Simulación de usuarios fríos

Para estudiar el arranque en frío (Schein et al., 2002) se retiró por completo del entrenamiento al 10 % de los usuarios (61 personas) y se evaluó la valoración **inmediatamente posterior** a sus primeras 0 o 3 valoraciones, como si fueran un formulario de bienvenida. Se repitió con tres muestras distintas. El peso del contenido en el fallback se eligió con una cuarta muestra de usuarios fríos disjunta de las de prueba.

Modelos

Líneas base

Para saber si un modelo aporta valor real se usaron cinco referencias: un recomendador **aleatorio**, la **media global** (solo para el error de calificación), la **popularidad por conteo** (lo más visto), la **popularidad suavizada** y los **sesgos de usuario e ítem** regularizados. La popularidad suavizada usa una media bayesiana:

$$\textit{puntuación} = n/(n + m) \cdot \textit{media del ítem} + m/(n + m) \cdot \textit{media global}$$

donde n es el número de valoraciones de la película y $m = 27$ es el percentil 90 de esos conteos. La Tabla 3 muestra por qué hace falta suavizar: ordenar por la media cruda pone arriba películas con un único voto de cinco estrellas, mientras que el orden suavizado solo admite títulos con cientos de valoraciones.

Tabla 3

Top-5 por media cruda frente a Top-5 por popularidad suavizada

Orden y película	Media	Valoraciones	Puntuación suavizada
Media cruda: Lamerica (1994)	5.00	2	3.605
Media cruda: Heidi Fleiss: Hollywood Madam (1995)	5.00	2	3.605
Media cruda: Lesson Faust (1994)	5.00	2	3.605
Suavizada: Shawshank Redemption, The (1994)	4.43	317	4.356
Suavizada: Godfather, The (1972)	4.29	192	4.192
Suavizada: Fight Club (1999)	4.27	218	4.188
Suavizada: Star Wars: Episode IV - A New Hope (1977)	4.23	251	4.160
Suavizada: Usual Suspects, The (1995)	4.24	204	4.152

Nota. Calculado con todas las valoraciones (LAB08). Fuente: *reports/popularity_top10.csv*.

Recomendador por contenido

Cada película se representó con un vector TF-IDF de sus géneros (Salton y Buckley, 1988) y se comparó con la similitud coseno (Lops et al., 2011). El perfil de una persona es la suma de los vectores de lo que valoró, ponderada por la diferencia entre cada calificación y su media personal: lo que le gustó más que su promedio acerca hacia esos géneros y lo que le gustó menos aleja. La similitud significa «comparte géneros registrados», no «le gustará».

Con solo 20 géneros posibles hay **empates masivos**. Para *Toy Story (1995)* las diez películas más similares tienen similitud 1.00 y comparten exactamente sus cinco géneros; para *The Silence of the Lambs (1991)* son diez títulos Crime|Horror|Thriller, tres de ellos de la saga *Saw*. Los empates se rompen por popularidad, una decisión documentada en el código. Es un ejemplo claro de sobre-especialización.

Factorización matricial

La señal colaborativa se obtuvo con una factorización matricial con sesgos entrenada por descenso de gradiente estocástico (Koren et al., 2009). La calificación estimada es la media global más un sesgo de usuario, un sesgo de ítem y el producto de dos vectores latentes. Se exploraron 10, 20 y 50 factores con 12 y 25 épocas, una tasa de aprendizaje de 0.01 y regularización 0.05. Una persona o película ausente del entrenamiento no tiene vector aprendido, de modo que la factorización no puede recomendar a un usuario nuevo.

Híbrido y política de fallback

El híbrido combina las señales normalizadas por usuario entre 0 y 1 (Burke, 2002). El parámetro α pondera la señal colaborativa frente a la de contenido, como en LAB09. Se añadió un segundo parámetro, γ, que mezcla esa combinación con la popularidad por conteo; con $\gamma = 0$ se recupera el híbrido original de LAB09. El motivo y el momento de ese cambio se explican en la sección de errores. La política por tipo de usuario es la siguiente:

- **Con historial y vector colaborativo:** híbrido con α y γ. La lista excluye siempre lo ya visto.
- **Sin vector colaborativo, con algunas valoraciones:** $\beta \cdot \text{contenido} + (1 - \beta) \cdot \text{popularidad}$.
- **Sin valoraciones:** popularidad por conteo, etiquetada explícitamente como **«no personalizada»**.

Evaluación y decisión

Métricas

Para el error de calificación se usaron RMSE y MAE. Para el ranking, con un único ítem relevante por persona, se calculó **HitRate@10** (proporción de usuarios cuyo ítem retenido aparece

en su Top-10), **NDCG@10** (que premia que aparezca más arriba; Järvelin y Kekäläinen, 2002), Precision@10 y MRR@10. Más allá del acierto (Kaminskas y Bridge, 2016) se midieron la **cobertura** del catálogo, la **cuota de cabeza** (posiciones ocupadas por el 10 % de películas más valoradas), la **novedad** ($-\log_2$ de la fracción de usuarios que vio el ítem), la diversidad intra-lista por géneros y el índice de Gini de la exposición. Cada resultado se acompaña de su error estándar binomial: con 610 usuarios y aciertos del orden del 4 %, una diferencia menor que 0.008 no se distingue del ruido.

Selección en validación y frontera de Pareto

En validación se evaluaron 93 configuraciones con tres semillas: las tres líneas base no personalizadas o de contenido y 90 combinaciones de factores, épocas, α y γ. La regla de decisión, aplicada solo con validación, fue: (1) frontera de Pareto, maximizando HitRate@10 y cobertura y minimizando el costo de cómputo (factores × épocas × valoraciones, una medida determinista) y el número de parámetros; (2) equivalencia en acierto, a no más de un error estándar de la mejor HitRate@10 (0.051 − 0.009), siguiendo la regla de un error estándar (Hastie et al., 2009); (3) equivalencia en cobertura, al menos el 90 % de la mayor cobertura del grupo anterior; y (4) entre las restantes, **la más barata**.

Tabla 4

Configuraciones equivalentes en validación y líneas base de referencia

Configuración	HitRate@10	Cobertura	Costo de cómputo	Elegida
Popularidad (conteo)	0.031	1.3 %	99 616	No
Popularidad suavizada	0.043	0.8 %	99 616	No
Contenido	0.005	21.8 %	185 098	No
MF 10 f. / 12 ép., α = 0.5, γ = 0.25	0.044	3.8 %	11 953 920	Sí
MF 20 f. / 12 ép., α = 0.5, γ = 0.25	0.043	3.9 %	23 907 840	No
MF 20 f. / 25 ép., α = 0.75, γ = 0.25	0.044	3.8 %	49 808 000	No
MF 50 f. / 25 ép., α = 0.75, γ = 0.25	0.044	4.1 %	124 520 000	No

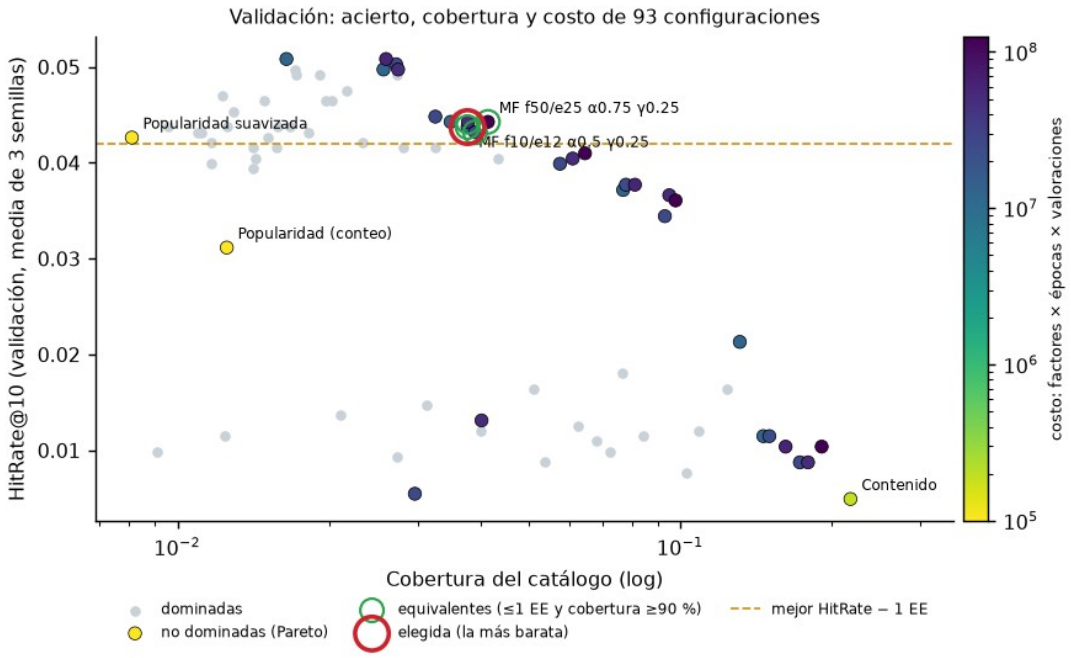
Nota. Media de tres semillas. Las cuatro configuraciones de MF están en la frontera de Pareto, a ≤ 1 EE de la mejor HitRate@10 y con cobertura equivalente. Fuente: *reports/validation_pareto.csv*.

La configuración elegida fue una **factorización de 10 factores y 12 épocas con α = 0.5 y γ = 0.25**. La alternativa de 50 factores y 25 épocas alcanzaba el mismo acierto con más de diez veces

el cómputo, por lo que no se eligió el modelo más grande. La Figura 1 muestra el compromiso: a medida que se gana cobertura se pierde acierto, y las configuraciones con prior de popularidad ocupan el codo de la curva.

Figura 1

Acierto, cobertura y costo de las 93 configuraciones evaluadas en validación

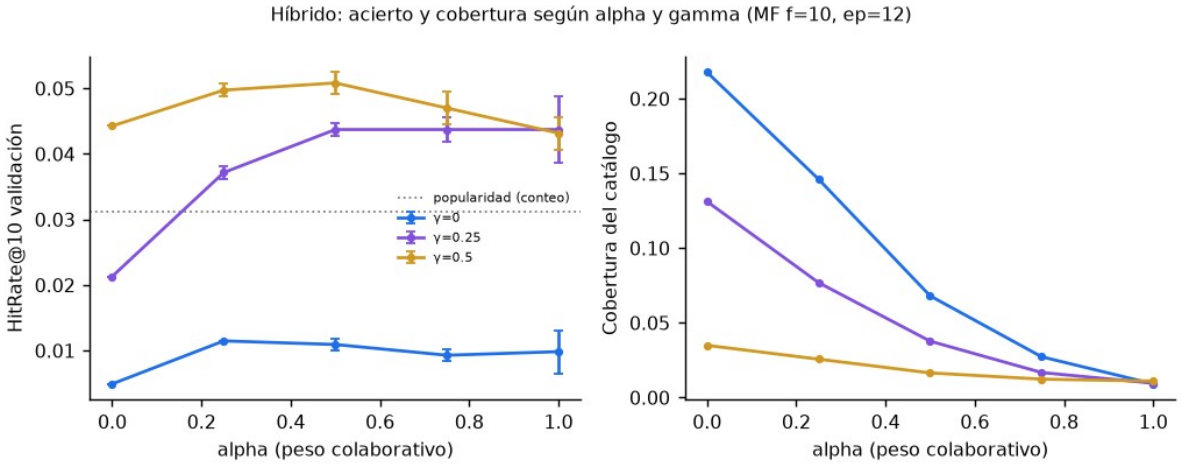


Nota. Los puntos coloreados son no dominados; el color indica el costo de cómputo. El círculo rojo marca la configuración elegida y la línea discontinua el umbral de un error estándar.

La Figura 2 aísla el efecto de los pesos del híbrido sobre la factorización elegida. Sin prior de popularidad ($\gamma = 0$), aumentar α reduce la cobertura de 21.8 % a 0.9 % y el acierto se queda entre 0.005 y 0.012, muy por debajo de la popularidad. Con $\gamma > 0$ y $\alpha \geq 0.25$ el acierto sube al nivel de la popularidad (0.037 a 0.051), pero la cobertura vuelve a caer a medida que crecen α o γ. El punto elegido, $\alpha = 0.5$ y $\gamma = 0.25$, conserva una cobertura tres veces mayor que la de la popularidad por conteo.

Figura 2

Efecto de α y γ sobre el acierto y la cobertura del híbrido en validación



Nota. Barras de error: desviación estándar entre semillas. Línea punteada: popularidad por conteo.

Resultados en el conjunto de prueba

La Tabla 5 resume la evaluación única en prueba sobre 610 usuarios. Es la tabla principal de resultados del ejercicio.

Tabla 5

Resultados en el conjunto de prueba (610 usuarios, media de tres semillas)

Modelo	RMSE	HitRate@10 ± EE	NDCG@10	Cobertura	Cuota cabeza	Novedad
Aleatorio	—	0.001 ± 0.001	0.000	46.3 %	9 %	7.47
Media global	1.117	—	—	—	—	—
Popularidad (conteo)	—	0.043 ± 0.008	0.019	1.3 %	100 %	1.42
Popularidad suavizada	1.058	0.034 ± 0.007	0.018	0.8 %	100 %	1.76
Sesgos usuario+ítem	0.973	0.023 ± 0.006	0.012	0.7 %	100 %	2.18
Contenido (TF-IDF)	—	0.008 ± 0.004	0.004	22.3 %	48 %	5.18
MF (10 factores, 12 épocas)	0.964	0.011 ± 0.004	0.006	0.9 %	62 %	3.92
Híbrido LAB09 (α = 0.25, γ = 0)	—	0.009 ± 0.004	0.005	14.4 %	41 %	4.93
Híbrido final (α = 0.5, γ = 0.25)	**—**	**0.031 ± 0.007**	**0.016**	**3.8 %**	**93 %**	**2.30**

Nota. EE: error estándar binomial entre usuarios. Cuota cabeza: posiciones ocupadas por el 10 % de películas más valoradas. Novedad en bits. La media global solo predice calificaciones. Fuente: *reports/test_results.csv*.

La popularidad por conteo obtuvo el mayor acierto (HitRate@10 = 0.043). El híbrido final quedó en 0.031, muy por encima de la factorización sola (0.011), del híbrido de LAB09 sin prior (0.009) y del contenido (0.008). A cambio de ese menor acierto puntual, el híbrido final cubre el 3.8 % del catálogo frente al 1.25 % de la popularidad, reduce la cuota de cabeza de 100 % a 93 % y eleva la novedad de 1.42 a 2.30 bits.

Comparación pareada

Como los modelos se evalúan sobre los mismos usuarios, la comparación correcta es pareada. Para cada semilla se calculó la diferencia de HitRate@10 usuario por usuario, un intervalo de confianza del 95 % por bootstrap (Efron y Tibshirani, 1993) y la prueba exacta de McNemar sobre los usuarios en los que solo uno de los dos modelos acierta (Dietterich, 1998). La Tabla 6 resume las tres semillas.

Tabla 6

Comparación pareada del híbrido final en prueba

Híbrido final frente a	Diferencia media de HitRate@10	IC 95 % más amplio entre semillas	p de McNemar (rango)
Popularidad (conteo)	-0.011	[-0.031, 0.010]	0.185 a 0.458
MF sola	+0.020	[0.005, 0.036]	0.002 a 0.013
Híbrido LAB09 ($\gamma = 0$)	+0.022	[0.008, 0.038]	0.000 a 0.002

Nota. IC por bootstrap de 5 000 remuestreos sobre usuarios; se muestra el límite inferior más bajo y el superior más alto de las tres semillas. Fuente: *reports/paired_tests.csv*.

Frente a la popularidad el intervalo incluye el cero en las tres semillas y la prueba de McNemar no es significativa: el acierto de ambos es estadísticamente indistinguible, aunque la estimación puntual del híbrido sea menor. Frente a la factorización sola y al híbrido de LAB09, en cambio, la mejora del híbrido final es clara y significativa.

Ratings frente a ranking

La factorización obtuvo el mejor RMSE (0.964), por delante de los sesgos de usuario e ítem (0.973), la popularidad suavizada (1.058) y la media global (1.117). Sin embargo, su HitRate@10 es la cuarta parte del de la popularidad. Predecir cuántas estrellas daría alguien a una película no equivale a predecir qué valorará después: una película poco vista puede recibir una predicción alta

y tener casi ninguna probabilidad de ser la siguiente. Es el mismo fenómeno documentado por Cremonesi et al. (2010) y la razón por la que la decisión se tomó con métricas de ranking y no con RMSE.

Usuarios con historial pequeño y amplio

Al dividir a los usuarios de prueba en tercios según el tamaño de su historial, el híbrido final acierta en el 3.3 % del tercio con historial pequeño (19 a 44 valoraciones), en el 5.1 % del tercio medio y solo en el 0.5 % del tercio con historial amplio (127 a 2 697 valoraciones). Quien ya valoró cientos de películas tiende a valorar después títulos poco populares, que son justamente los que el modelo casi nunca propone.

Usuarios fríos

Tabla 7

Resultados con usuarios fríos simulados (61 usuarios por semilla)

Escenario y modelo	HitRate@10	DE entre semillas	Cobertura	Fuente declarada
0 valoraciones: aleatorio	0.000	0.000	6.1 %	—
0 valoraciones: popularidad (conteo)	0.142	0.019	0.1 %	—
0 valoraciones: híbrido final → fallback	0.142	0.019	0.1 %	popularidad (no personalizada)
3 valoraciones: popularidad (conteo)	0.148	0.016	0.1 %	—
3 valoraciones: contenido	0.011	0.019	4.6 %	—
3 valoraciones: híbrido final → fallback (β = 0.5)	0.109	0.038	0.9 %	contenido + popularidad

Nota. Se evalúa la valoración inmediatamente posterior al onboarding. La factorización no tiene vector para ninguno de los usuarios fríos. Fuente: *reports/cold_start.csv*.

Sin valoraciones, el híbrido cae a la popularidad y lo declara; acierta en el 14.2 % de los casos porque las primeras películas que valora una persona nueva suelen ser muy populares. Con tres valoraciones, el fallback de contenido y popularidad logra 0.109, por debajo de la popularidad pura (0.148). El $\beta = 0.5$ elegido con la muestra fría disjunta no se sostuvo en prueba; con solo 61 usuarios por muestra la varianza es alta. Es la debilidad más concreta del sistema y el siguiente experimento a realizar.

Tres perfiles

Tabla 8

Top-5 recomendado por el híbrido final para tres perfiles

Posición	Historial amplio (usuario 414, 2 697 val.)	Historial pequeño (usuario 53, 19 val.)	Usuario nuevo (0 val.)
1	12 Angry Men (1957)	Pulp Fiction (1994)	Forrest Gump (1994)
2	Guess Who's Coming to Dinner (1967)	Forrest Gump (1994)	Shawshank Redemption (1994)
3	Seventh Seal (1957)	Shawshank Redemption (1994)	Pulp Fiction (1994)
4	Hustler (1961)	Fight Club (1999)	Matrix (1999)
5	Secrets & Lies (1996)	American Beauty (1999)	Silence of the Lambs (1991)

Nota. Ninguna lista contiene títulos ya vistos. La lista del usuario nuevo es no personalizada. Fuente: *reports/profiles.csv*.

Las listas cambian con el historial. El usuario de historial amplio recibe dramas clásicos acordes con sus valoraciones; el de historial pequeño, clásicos de los años noventa, todavía cerca de la popularidad; y el usuario nuevo, el Top de popularidad. Para el usuario de historial amplio, los pesos $\alpha = 0.25$ y $\alpha = 0.75$ sin prior producen el mismo Top-10. No es un error: su perfil de contenido apunta a Drama y 1 053 películas, el 10.8 % del catálogo, empatan en la similitud máxima, por lo que el contenido actúa como filtro de género y la señal colaborativa ordena dentro de ese grupo.

Costo computacional (Green AI)

El modelo final se entrena en unos 8 a 9 segundos en CPU con Python puro y tiene unos 114 000 parámetros en la factorización más una matriz TF-IDF de 185 000 valores. La popularidad se calcula en menos de una centésima de segundo y ocupa 9 742 valores, por lo que también pertenece a la frontera de Pareto: si el único objetivo fuera HitRate@10, sería una elección legítima. El protocolo completo, con las 93 configuraciones, la prueba y los usuarios fríos, tarda unos 11 minutos. La decisión usó un costo determinista en lugar del tiempo de reloj, que depende de la carga de la máquina.

Errores corregidos y límites

Errores detectados durante el desarrollo

El desarrollo no fue lineal y conviene dejar trazabilidad de los errores, porque varios cambiaron la decisión final:

- **Líneas base perdidas al agregar.** Una agrupación descartaba las filas con α vacío, que son precisamente las líneas base; no aparecían en la tabla de Pareto. Se detectó por un error de clave y se corrigió.
- **Objetivo de usuario frío poco realista.** La primera versión evaluaba la última valoración de cada usuario frío, a veces años después de su llegada. Se cambió a la valoración siguiente al onboarding.
- **Regla de decisión sesgada (protocolo v1).** Solo dejaba competir a híbridos y factorización y eligió un híbrido con HitRate@10 de prueba de 0.010 frente a 0.043 de la popularidad. Se rediseñó: compiten todos los modelos, se añadió el prior γ y se decide por Pareto, error estándar y cobertura.
- **Regla que premiaba el modelo más grande (v2a).** Elegía la mayor cobertura y usaba segundos de reloj como costo; seleccionó la factorización de 50 factores y 25 épocas. Se cambió a un costo determinista, tolerancia de cobertura y la opción más barata.

Límite metodológico declarado. Los cambios de la regla y la introducción de γ se hicieron después de ver resultados de prueba de versiones anteriores, aunque cada selección se calculó solo con validación. La prueba, por tanto, no está totalmente intacta. Los resultados de las versiones v1 y v2a se conservan en el repositorio para que la trayectoria pueda auditarse.

Límites del estudio

- **Un único ítem relevante por usuario y 610 usuarios:** las diferencias menores que 0.008 en HitRate@10 son ruido.
- **Contenido pobre:** 20 géneros producen empates masivos y sobre-especialización; no se usaron etiquetas, sinopsis ni elenco.

- **Sesgo de selección:** solo se observan películas que cada persona eligió ver y valorar; la ausencia no es rechazo.
- **Sin demografía:** no puede afirmarse nada sobre equidad por grupos de población.
- **Marcas de tiempo:** indican cuándo se valoró, no cuándo se vio; muchas cuentas valoran decenas de películas en el mismo segundo.
- **Dataset de desarrollo:** los números valen para la huella SHA-256 registrada.

Riesgos y mitigación

El riesgo principal es el **dominio de la popularidad y la retroalimentación**. El 93 % de las posiciones recomendadas por el híbrido final pertenece al 10 % de películas más valoradas y el índice de Gini de la exposición es 0.99. Si el sistema se desplegara, lo recomendado se vería más, generaría más valoraciones y reforzaría la concentración en el siguiente entrenamiento (Abdollahpouri et al., 2017). Como mitigación se propone mantener γ acotado, monitorizar la cobertura y la cuota de cabeza, reservar posiciones de exploración y reentrenar periódicamente con corte temporal. Las limitaciones y riesgos se documentaron además en una Dataset Card y una System Card, siguiendo la idea de documentar datos y modelos para su uso responsable (Gebru et al., 2021; Mitchell et al., 2019).

Pruebas, reproducibilidad y repositorio

El código está organizado como un paquete en *src/inf8239_u03* con módulos separados para datos, particiones, popularidad, contenido, factorización, híbrido, métricas, evaluación y el protocolo completo. Un único comando, *python scripts/run_experiments.py*, reproduce todas las tablas y figuras en *reports/*. El notebook ejecutado llama a ese mismo protocolo y añade la interpretación.

La carpeta *tests* contiene **36 pruebas automáticas** que se ejecutan en unos cinco segundos con datos sintéticos del mismo esquema que MovieLens, sin necesidad de descargarlo. Verifican, entre otras cosas, la ausencia de fuga temporal, que las particiones sean exactas, que no se recomiende lo ya visto, que el fallback se declare como no personalizado, las métricas contra casos

calculados a mano, la definición de dominancia de Pareto y el determinismo de la factorización con semilla.

La reproducibilidad se comprobó de tres maneras: el notebook se ejecutó completo tres veces y las métricas coincidieron exactamente, salvo los tiempos de reloj; la factorización reentrenada con la misma semilla produce parámetros idénticos; y un clon limpio del repositorio, instalado solo desde *requirements.txt*, pasa todas las pruebas y reproduce la huella del dataset. El entorno queda fijado con *requirements.txt* y *uv.lock* (Python 3.12).

Declaración de uso de inteligencia artificial

Se utilizó **Claude** (Anthropic), modelo Claude Opus 5.5, en un entorno de ejecución en la nube. La herramienta redactó el código de los módulos, las pruebas automáticas, el notebook y los borradores de la documentación, y ayudó a diagnosticar los errores descritos en la sección anterior. Los commits del repositorio la identifican como coautora.

Los prompts relevantes fueron el enunciado completo del Ejercicio 05 junto con las guías de LAB08 y LAB09, con la instrucción de consolidarlos en un producto ejecutable y defendible; la entrega del archivo de MovieLens descargado manualmente, porque el entorno de la herramienta no tenía acceso de red a GroupLens; y la solicitud de adaptar este informe al formato institucional.

Las verificaciones realizadas fueron la ejecución de las 36 pruebas, la auditoría del dataset con su huella, el determinismo con semilla, la re-ejecución completa del notebook, la comparación pareada para no afirmar diferencias que son ruido y la comprobación de que todas las cifras de este documento provienen de los archivos de *reports/*. Cada referencia bibliográfica se contrastó con su fuente o con un índice bibliográfico. Las correcciones aplicadas son las descritas en la sección de errores. El estudiante revisó, ejecutó y responde por todo el código, los resultados, las referencias y las conclusiones entregadas.

Conclusiones

Luego de completar el ejercicio se comprobó que, en MovieLens con un corte temporal por usuario, la popularidad es una línea base difícil de superar. La factorización matricial obtuvo el mejor error de calificación y, sin embargo, fue de las peores ordenando lo que cada persona valoró después; el híbrido de LAB09, que combina factorización y contenido, tampoco superó a la popularidad. Mejor RMSE no implica mejor Top-10.

Dos resultados fueron especialmente útiles para el aprendizaje. El primero es que medir solo el acierto conduce a recomendar siempre lo mismo: la popularidad acierta, pero cubre el 1.25 % del catálogo y todas sus posiciones pertenecen a la cabeza. El segundo es que fijar la regla de decisión antes de mirar la prueba es tan importante como el modelo: las dos versiones previas de la regla eligieron configuraciones peores o innecesariamente costosas, y corregirlas dejó una huella metodológica que se declara.

La conclusión accionable es la siguiente: para usuarios con historial se adopta el **híbrido final** (factorización de 10 factores y 12 épocas, $\alpha = 0.5$, $\gamma = 0.25$), cuyo acierto es estadísticamente indistinguible del de la popularidad, que triplica la cobertura del catálogo, reduce la concentración y personaliza las listas. Para usuarios nuevos se usa popularidad por conteo, declarada como no personalizada. Con tres valoraciones de bienvenida, la evidencia sugiere reducir β o usar popularidad pura, lo que debe validarse con más usuarios fríos.

El principal riesgo identificado es el dominio de la popularidad y su retroalimentación. Como trabajo futuro se propone enriquecer el contenido con etiquetas y sinopsis, añadir diversificación del Top-10, ajustar el fallback de bienvenida con una muestra mayor y evaluar con un corte temporal global. En sentido general, el ejercicio permitió integrar los contenidos de la unidad: gobernanza y auditoría de datos, comparación rigurosa de enfoques generativos y colaborativos, métricas más allá del acierto, costo computacional y la construcción de un producto reproducible que otras personas pueden ejecutar y revisar.

Referencias

Datos y sistemas de recomendación

Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction, 12*(4), 331–370. https://doi.org/10.1023/A:1021240730564

Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems, 5*(4), Artículo 19. https://doi.org/10.1145/2827872

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer, 42*(8), 30–37. https://doi.org/10.1109/MC.2009.263

Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. En F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 73–105). Springer.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523.

Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 253–260. https://doi.org/10.1145/564376.564421

Evaluación y comparación de modelos

Cremonesi, P., Koren, Y., & Turrin, R. (2010). Performance of recommender algorithms on top-N recommendation tasks. *Proceedings of the Fourth ACM Conference on Recommender Systems*, 39–46. https://doi.org/10.1145/1864708.1864721

Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation, 10*(7), 1895–1923.

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2.ª ed.). Springer.

Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems, 20*(4), 422–446.

Kaminskas, M., & Bridge, D. (2016). Diversity, serendipity, novelty, and coverage: A survey and empirical analysis of beyond-accuracy objectives in recommender systems. *ACM Transactions on Interactive Intelligent Systems, 7*(1), Artículo 2. https://doi.org/10.1145/2926720

Sesgos, documentación y Green AI

Abdollahpouri, H., Burke, R., & Mobasher, B. (2017). Controlling popularity bias in learning-to-rank recommendation. *Proceedings of the Eleventh ACM Conference on Recommender Systems*, 42–46. https://doi.org/10.1145/3109859.3109912

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM, 64*(12), 86–92.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–229.

Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2020). Green AI. *Communications of the ACM, 63*(12), 54–63.

Apéndice

Repositorio del proyecto

El código fuente, el notebook ejecutado, las pruebas, los reportes y la documentación se encuentran disponibles en el repositorio de GitHub del proyecto:

https://github.com/melvinrivas03/inf8239-u03-e05-recomendador

Para reproducir los resultados: instalar el entorno con *requirements.txt*, obtener MovieLens con *python scripts/download_data.py*, verificar con *python scripts/audit_data.py*, ejecutar *python -m pytest* y luego *python scripts/run_experiments.py*. El detalle está en el README.

Figura A1

Código QR de acceso al repositorio del proyecto